

Machine Learning Technologies Project - Garbage Classification

Matthew McCarthy - (G00373167)

Bachelor of Science (Honours) in Computing in Software Development, ATU Galway

Abstract

This paper investigates the use of convolutional neural networks for classifying household waste images into ten categories using the Garbage Classification dataset (12,259 images, 384×384 RGB). Six CNN architectures were developed iteratively from scratch, exploring the effects of network depth, batch normalisation, dropout, data augmentation, greyscale conversion, and image resolution. Transfer learning with VGG16 was then applied, followed by fine-tuning. The best-performing model, a fine-tuned VGG16, achieved 92.37% validation accuracy and a macro F1-score of 0.9167, rising from 74.54% and 0.7443 for the best scratch model. On the held-out test set, the final model reached 91.57% accuracy with an inference speed of approximately 2 milliseconds per image. Testing on original photographs revealed challenges with domain shift, particularly when objects did not fill the frame.

1 Introduction

Waste management and recycling are among the significant environmental challenges the world faces, with improper sorting of household waste reducing the efficiency of recycling systems. Automated image classification offers a potential solution by identifying waste categories from photographs, enabling smarter sorting at scale. This paper investigates the use of convolutional neural networks (CNNs) for classifying images of household waste.

The Garbage Classification dataset was selected for this task, containing over 12,000 images of common household waste. The dataset presents a genuine classification challenge due to real-world visual variability and similarities between certain categories.

A series of CNN architectures were built from scratch, beginning with a simple shallow network and iteratively improving through the addition of convolutional layers, dropout regularisation, and data augmentation. The effects of greyscale conversion and reduced image resolution were also investigated. Transfer learning was then applied using VGG16 as a base model, followed by fine-tuning of the upper layers. All models were compared using macro-averaged F1-score as the primary metric to account for class imbalance, alongside validation accuracy, training time, parameter count, and inference speed. The best-performing model was evaluated on a held-out test set and tested on original photographs taken by the author.

2 Data and Preprocessing

2.1 Dataset Description

The Garbage Classification dataset is derived from the Garbage Dataset (GD), a publicly available benchmark for automated waste segregation [1]. It contains exactly 12,259 images across ten categories: battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes, and trash. All images are 384×384 pixels in RGB colour. The images were collected through a mobile application and curated web sources. They exhibit realistic variability in lighting, backgrounds, and object occlusion, making the classification task highly authentic.

2.2 Exploratory Analysis

The class distribution was examined to understand the balance of the dataset. A significant imbalance was observed, with the largest class (clothes) containing 1,892 images and the smallest (trash) containing just 453. This imbalance means that accuracy alone would be a misleading metric, as a model predicting only the majority class could still achieve a deceptively high score. Consequently, the macro-averaged F1-score was adopted for model comparison, as it treats all classes equally regardless of their size. Figure 1 illustrates the class distribution.

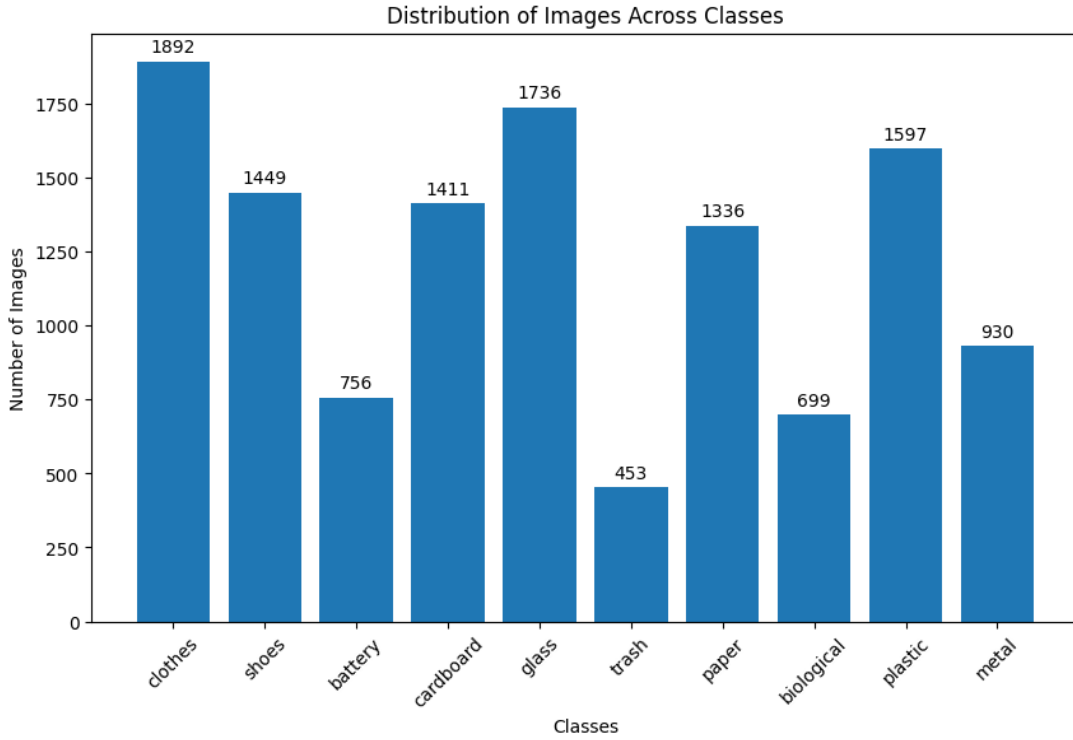


Figure 1: Distribution of images across the ten waste categories.

Sample images from each category were also inspected to verify pixel value ranges. Pixel values ranged from 0 to 255 in most categories, confirming that normalisation would be required before training.

2.3 Preprocessing and Splits

The dataset was loaded using the Keras image dataset utility with a fixed random seed of 373167 (derived from the student ID) to ensure reproducibility. Images were kept at their native resolution and loaded in batches of 32. The data was split into 70% training and 30% holdout. The holdout set was further divided into two-thirds test (20% of total) and one-third validation (10% of total). The validation set was used exclusively for model selection and hyperparameter decisions, while the test set was reserved for final evaluation of the chosen model only. All splits were cached and prefetched to maximise GPU utilisation during training. To address the class imbalance identified earlier, class weights were computed as $\frac{\text{total samples}}{\text{number of classes} \times \text{class count}}$, giving higher weight to under-represented categories such as trash and battery. These weights were passed to the training process for every model, penalising misclassification of minority classes more heavily. Pixel values were normalised to the $[0, 1]$ range by rescaling by a factor of $\frac{1}{255}$ within each model architecture.

3 Experiments

3.1 CNN from Scratch

Six CNN architectures were developed iteratively, each building on lessons from the previous model. All models used the Adam optimiser, sparse categorical cross-entropy loss, class weighting, and early stopping with a patience of five epochs (restoring the best weights). Early stopping served as an additional regularisation measure, halting training when validation loss ceased to improve and preventing the model from overfitting to the training data. A maximum of 50 epochs was set for all training runs. Global average pooling was used in place of a flatten layer to significantly reduce parameter counts.

Model 1 (Shallow Baseline): A minimal network with two convolutional layers (32 and 64 filters) each followed by max pooling, and a single 64-unit dense layer. This achieved a validation accuracy of 53.98% and a macro F1 of 0.5375, establishing a baseline.

To build upon this baseline, **Model 2** introduced two additional convolutional layers (128 and 256 filters) to increase the network’s capacity to learn more complex features. Validation accuracy rose to 66.27% (F1: 0.6524), confirming that greater depth improved feature extraction.

To combat overfitting, **Model 3** added batch normalisation after each convolutional layer, along with dropout (0.5 and 0.3) before the dense layers. Surprisingly, this performed worse than Model 2 at 52.61% (F1: 0.5215). The combination of batch normalisation and dropout appeared to conflict, with batch normalisation normalising activations that dropout was simultaneously disrupting.

To isolate the cause, **Model 4** removed dropout and kept only batch normalisation. Performance improved over Model 3 to 61.37% (F1: 0.6054) but still fell short of Model 2, suggesting batch normalisation alone was not sufficient for this dataset.

Conversely, **Model 5** removed batch normalisation and retained only dropout. This was the best-performing scratch model at 74.54% (F1: 0.7443), indicating that dropout was the more effective regularisation strategy for this task.

Finally, **Model 6** applied random horizontal flips, rotations ($\pm 10\%$), and zoom ($\pm 10\%$) to Model 5. Performance dropped to 65.46% (F1: 0.6485). The augmentation likely introduced too much variation for the relatively small network to learn from within 50 epochs.

Greyscale Comparison: Model 5 was retrained on greyscale images to assess the importance of colour information. Validation accuracy fell to 68.51% (F1: 0.6770), a drop of six percentage points, confirming that colour is a useful discriminative feature for waste classification.

Image Size Comparison: Model 5 was retrained at 224×224 resolution (down from 384×384) to evaluate the trade-off between detail and training speed. Performance was nearly identical at 74.14% (F1: 0.7333) with significantly faster training (4 minutes versus 10 minutes), suggesting the higher resolution offered minimal benefit for the scratch CNN. The 224×224 resolution was adopted for transfer learning as it is the native input size for VGG16.

3.2 Transfer Learning

VGG16, pre-trained on ImageNet, was used as a feature extractor. The convolutional base was frozen and a custom classification head was added: global average pooling, dropout (0.5), a 64-unit dense layer, dropout (0.3), and a softmax output layer. Images were preprocessed using VGG16’s expected input format rather than simple rescaling. This model achieved 88.19% validation accuracy (F1: 0.8784) with only 4 minutes 51 seconds of training, a substantial improvement over every scratch model.

Fine-tuning was then performed by unfreezing the top four layers of the VGG16 base and retraining with a reduced learning rate of 10^{-5} for 15 epochs. The low learning rate was chosen to prevent catastrophic forgetting of the pre-trained weights, allowing the upper convolutional filters to adapt to the garbage dataset while preserving the general features learned from ImageNet. The fine-tuned model reached 92.37% validation accuracy (F1: 0.9167), the best result of all experiments.

4 Results and Comparison

Table 1 summarises the validation performance, training time, and parameter count for all models. The fine-tuned VGG16 achieved the highest validation accuracy (92.37%) and macro F1-score (0.9167). Among the scratch models, Model 5 (dropout only) was the clear winner at 74.54% accuracy and 0.7443 F1.

Model	Val Acc	Macro F1	Train Time	Parameters
M1: Shallow (2 conv)	53.98%	0.5375	7m 33s	24,202
M2: Deep (4 conv)	66.27%	0.6524	6m 17s	405,514
M3: BN + Dropout	52.61%	0.5215	3m 21s	407,434
M4: BN Only	61.37%	0.6054	4m 03s	407,434
M5: Dropout Only	74.54%	0.7443	9m 55s	405,514
M6: Dropout + Aug	65.46%	0.6485	10m 01s	405,514
M5 Greyscale	68.51%	0.6770	14m 05s	404,938
M5 at 224×224	74.14%	0.7333	3m 57s	405,514
VGG16 Transfer	88.19%	0.8784	4m 51s	14,748,170
VGG16 Fine-Tuned	92.37%	0.9167	2m 11s	14,748,170

Table 1: Validation results, training time, and parameter count for all models.

The fine-tuned VGG16 was selected as the final model and evaluated on the held-out test set, achieving 91.57% accuracy and a macro F1-score of 0.9074. The slight drop from validation to test performance is expected and indicates that the model generalises well to unseen data without significant overfitting.

The confusion matrix (Figure 2) revealed that the model’s most common errors involved visually similar categories. Plastic and trash were the most frequently confused pair, which is understandable given that many trash items are in fact plastic-based. Paper and cardboard also showed some cross-prediction, likely due to their similar textures and colours. Biological waste was occasionally misclassified as other categories, possibly due to the wide visual variety within that class. Despite class weighting, per-class F1-scores varied considerably: well-represented and visually distinct categories such as clothes (0.98) and shoes (0.97) performed strongly, while trash (0.79) and metal (0.86) remained weaker due to their visual overlap with other categories.

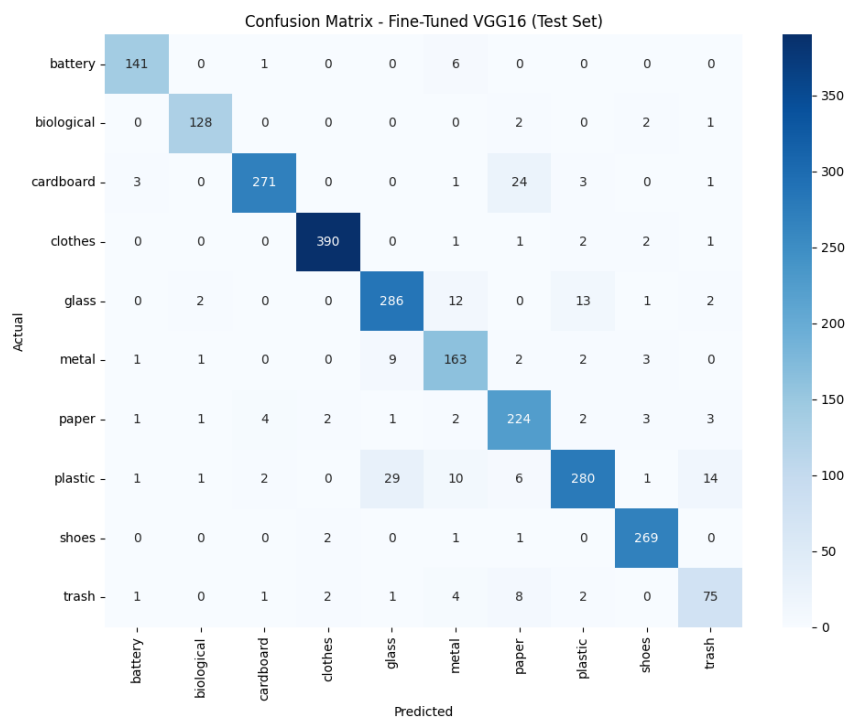


Figure 2: Confusion matrix for the fine-tuned VGG16 on the held-out test set.

In terms of efficiency, the transfer learning approach achieved dramatically better results than any scratch model while requiring comparable or less training time. The VGG16-based models contain 14.7 million parameters compared to approximately 405,000 for the scratch models, yet the frozen base required no gradient computation during initial training. Inference on the full test set took approximately 5.3 seconds, averaging around 2.2 milliseconds per image, which is practical for real-time applications.

5 Real-World Test

To evaluate the model beyond the dataset, two sets of four photographs were taken by the author of real household waste items. Each photograph included a handwritten note with the author's name and student ID.

The first set of images was taken from a distance: a plastic milk carton, crumpled paper, a battery, and a shoe. The model correctly identified only the paper (35.5% confidence), misclassifying the milk carton as biological (84.3%), the battery as biological (59.7%), and the shoe as paper (82.5%). A second set was taken at closer range with different items: a glass perfume bottle, a plastic zip bag, a red paper envelope, and a pair of shoes. Again, only one was correct — the paper envelope (91.0% confidence) — with the glass bottle misclassified as biological (85.7%), the zip bag as paper (87.8%), and the shoes as biological (93.4%).

Several factors likely contributed to the poor performance. The most significant was that the objects did not fill the frame in the way the training images do. Training images in the dataset are tightly cropped around the object, whereas the author's photographs included substantial background. The model also had to contend with lighting and environments not represented in the training data. It is notable that the model defaulted to "biological" for many misclassifications, which may reflect the visual diversity within that training category acting as a catch-all. Some of these misclassifications align with patterns observed in the confusion matrix; for example, plastic was frequently confused with glass and trash on the test set, and the model's tendency to predict biological is consistent with that category's broad visual range causing it to absorb ambiguous predictions.

Despite the low accuracy on personal photographs, the model achieved over 91% on the held-out test set, suggesting the gap is primarily a domain shift problem rather than a fundamental model weakness. A production system would benefit from tighter cropping or object detection as a preprocessing step.

6 Conclusion

This paper explored CNN-based image classification for household waste sorting using the Garbage Classification dataset. Through iterative development of six scratch architectures, dropout was found to be the most effective regularisation technique, while the combination of batch normalisation and dropout proved counter-productive. Colour information contributed meaningfully to classification accuracy, whereas reducing image resolution from 384×384 to 224×224 had negligible impact on performance while halving training time.

Transfer learning with VGG16 dramatically outperformed all scratch models, and fine-tuning the upper layers further improved the macro F1-score from 0.8784 to 0.9167 on the validation set. The final model achieved a test accuracy of 91.57% and a macro F1 of 0.9074, with an inference speed of approximately 2 milliseconds per image. However, testing on personal photographs revealed a significant domain shift: the model struggled with images where the object did not dominate the frame, highlighting the gap between curated dataset performance and real-world deployment.

Future work could address this through object detection or segmentation as a preprocessing step, ensuring the classifier receives tightly cropped inputs regardless of how the photograph was taken.

A AI Tools

AI tools (Claude via Claude Code and Google Gemini) were used throughout this project for code assistance and paper review. Below are specific examples of what was asked, what was produced, and what was changed.

Data splitting: The 70/10/20 train/validation/test split was not covered in the module labs. Claude was asked how to implement this. Its first suggestion caused data leakage by not properly isolating the splits. After finding a correct approach from an online source, the author asked Claude to verify it, and the corrected version was adopted.

EarlyStopping callback bug: Claude identified that reusing a single EarlyStopping callback object across multiple models caused the internal state (best weights, patience counter) to carry over between training runs. This meant `restore_best_weights` could restore the previous model's weights instead of the current one's. The fix was to create a helper function that returns a fresh callback object for each training run.

Paper drafting: Claude was used to generate initial drafts of each section of this paper based on the results and methodology from the notebook. The author reviewed, edited, and rewrote sections to ensure accuracy and to match their own writing style. Google Gemini was also used to review draft sections and suggest improvements, some of which were adopted (e.g. smoother transitions between model descriptions) and others rejected (e.g. adding the term “ablation study” and expanding theoretical explanations beyond what the brief required).

References

- [1] Suman Kunwar. The garbage dataset (gd): A multi-class image benchmark for automated waste segregation, 2026.